



ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

The State of Vue

Chapter 9

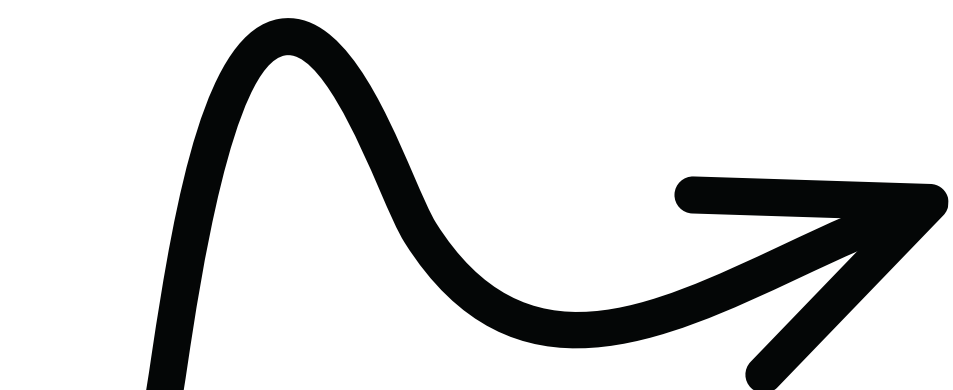
Advanced Web Design



Intro

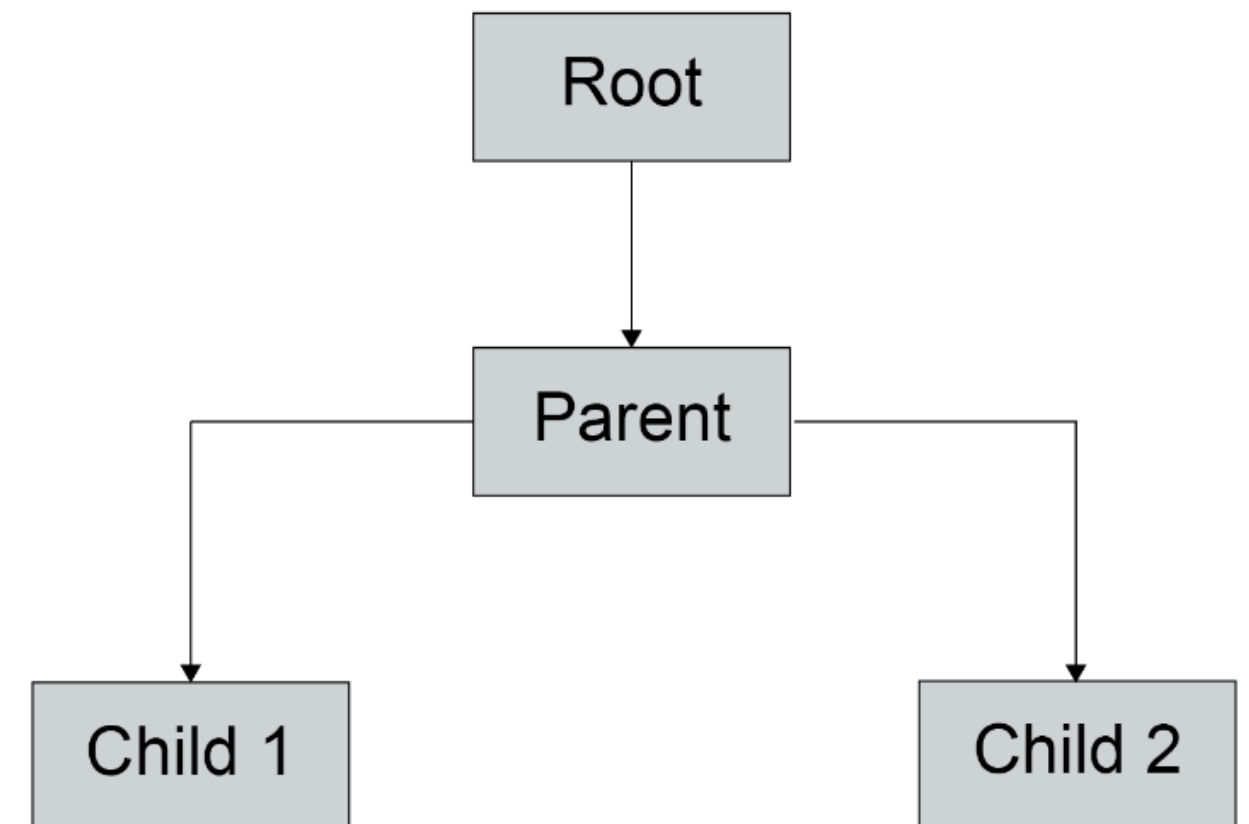
Here, you'll begin by taking a look at how problems with states arise, how state management can help address it, and what features Vue.js 3 has to help you deal with it directly.

You'll learn this while building a simple **profile card application** that uses **multiple components** between which the state needs to be synchronized.

- 
- Understanding the component architecture and the problem of the state
 - Holding the state in a common ancestor component
 - Adding simple state management
 - Deciding when to use a local state or global state

Understanding the component architecture and the problem of the state

- In previous chapters, we have seen how to use a **local state** and **props** to hold the state and share it in a parent-child component hierarchy.
- Now, we will begin by showing how to leverage **state**, **props**, and events to share states between components that do not have a parent-child configuration. These types of components are called **siblings**.

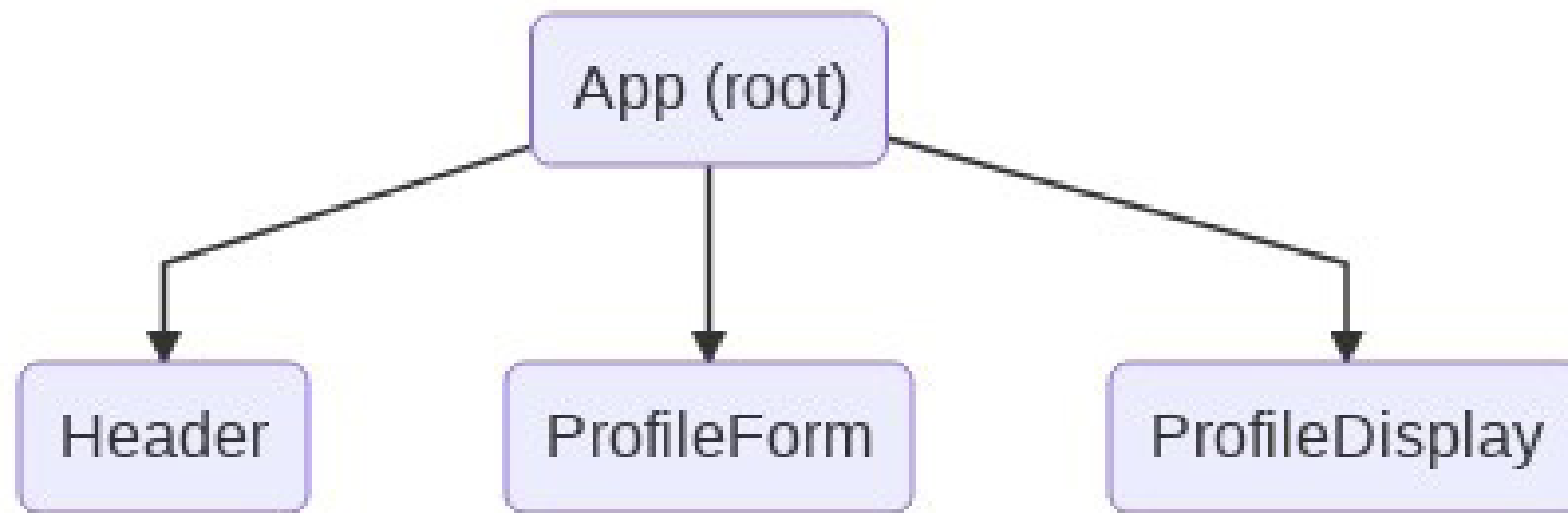


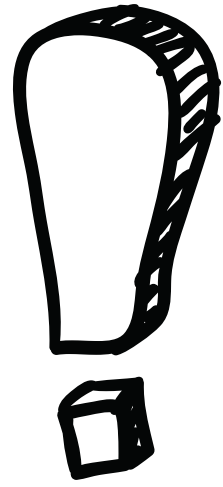
Given that we want to build a profile card generator, we can break the application down into three sections:

a **header**, where we will have global controls and display the title of the page;

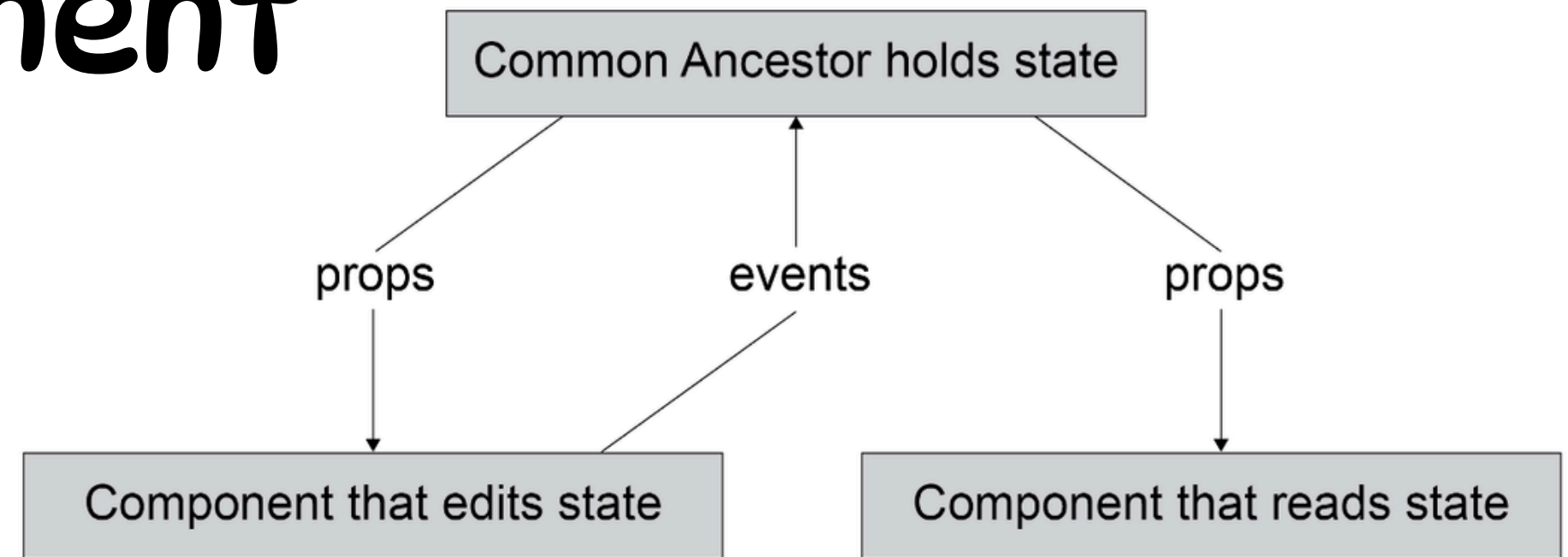
a **profile form**, where we will capture data; and finally,

a **profile display**, where we will display the profile card.





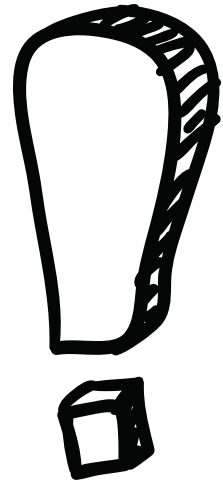
Holding the state in a common ancestor component



To only hold the state with the state component and props, and update it with events, we will store it in the **nearest common ancestor component**.

state is only propagated through **props** and is only updated through **events**. In this case, all the state components will live in a shared ancestor of the components that require them.

The **App** component, since it is the root component, is a good default for **holding a shared state**.

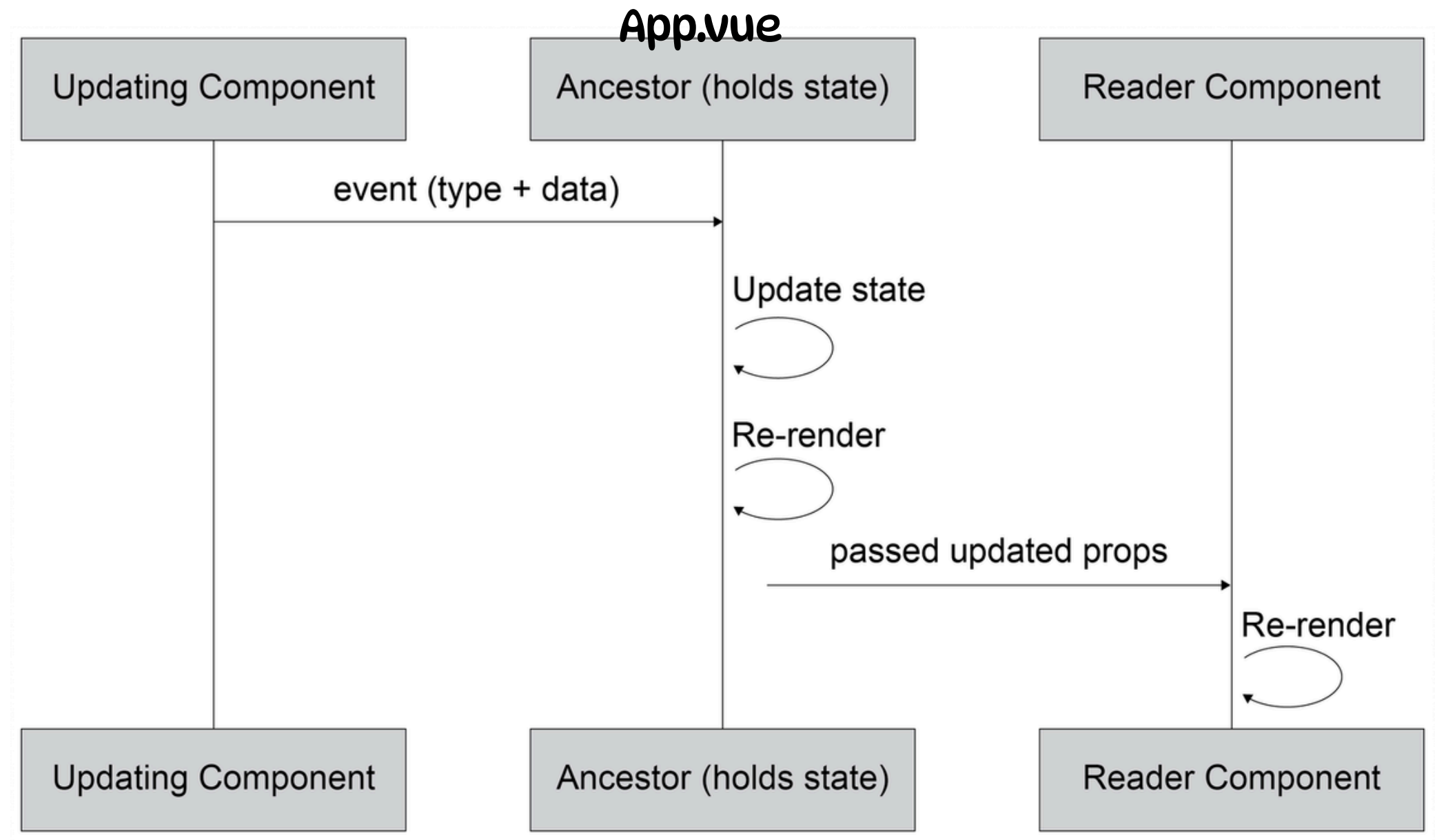


Holding the state in a common ancestor component

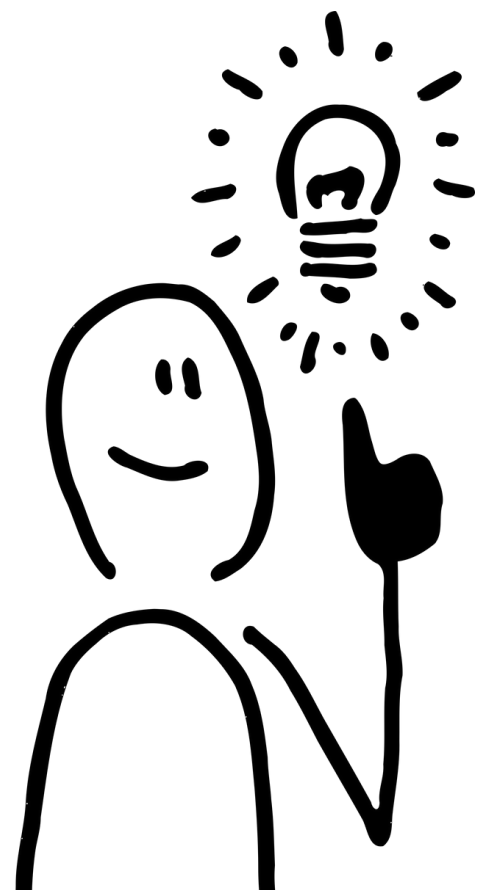
To change state, a component needs to **emit events** up to the component holding our state (the shared ancestor).

The shared ancestor needs to **update state** according to the **data and type of events**.

This, in turn, causes a **re-render**, during which the ancestor component **passes the updated props** to the component reading state.



Example: profile card editor



AppHeader.vue

```
<template>
  <header class="w-full block p-4 border-b bg-blue-300
border-gray-700">
    <h2 class="text-xl text-gray-800">Profile Card Generator</h2>
  </header>
</template>
```

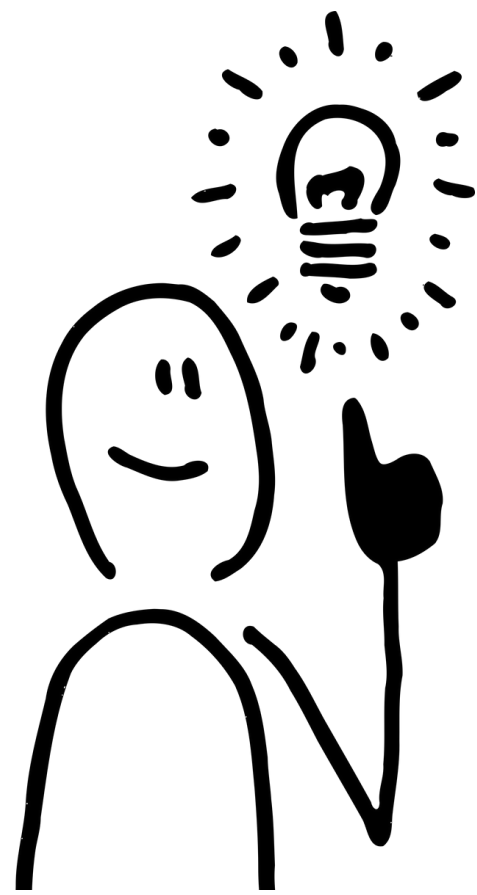
App.vue

```
<script setup>
import AppHeader from '@components/AppHeader.vue'
</script>

<template>
  <div id="app">
    <AppHeader/>
  </div>
</template>
```

Profile Card Generator

Example: profile card editor



AppProfileForm.vue

```
<template>
  <section class="md:w-2/3 flex flex-col
p-12 items-center bg-red-200">
    <!-- Inputs -->
  </section>
</template>
```

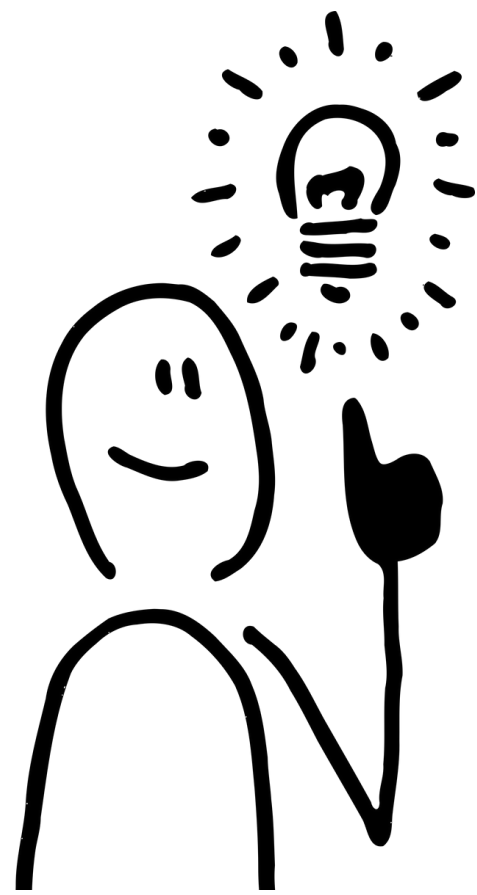
We will similarly create an AppProfileForm file; the purpose of this component is to lay out the labels and form fields for editing the profile

AppProfileDisplay.vue

```
<template>
  <section class="md:w-1/3 h-64 bg-blue-200 flex">
    <!-- Profile Card -->
  </section>
</template>
```

We will then create an AppProfileDisplay file; this component handles displaying the profile so the user can preview their edits

Example: profile card editor

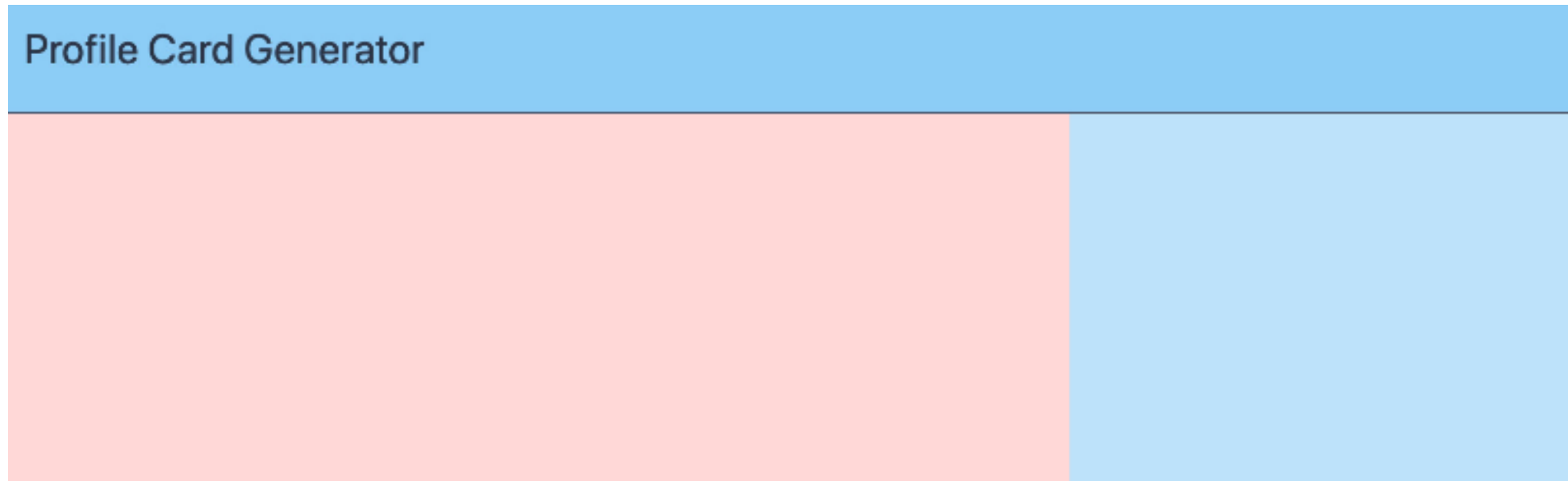


Both of our containers (AppProfileForm and AppProfileDisplay) can now be imported and rendered in **App.vue**:

```
<script setup>
import AppHeader from '@components/AppHeader.vue'
import AppProfileDisplay from '@components/AppProfileDisplay.vue'
import AppProfileForm from '@components/AppProfileForm.vue'
</script>
```

```
<template>
  <div id="app">
    <AppHeader/>
    <div class="flex flex-col md:flex-row">
      <AppProfileForm />
      <AppProfileDisplay />
    </div>
  </div>
</template>
```

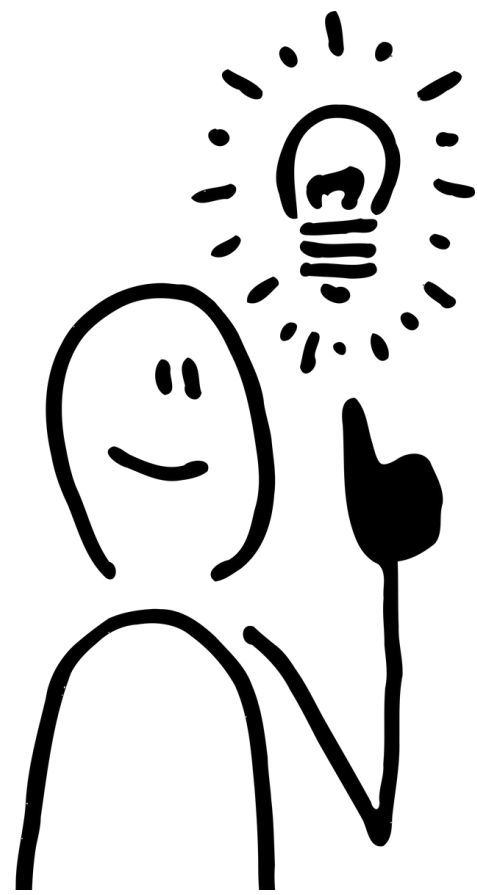
Profile Card Generator



Example: profile card editor

To add a form field, in this case, name, we will start by adding an input to **AppProfileForm**:

```
<template>
  <section class="md:w-2/3 h-64 bg-red-200 flex
flex-col p-12 items-center">
    <!-- Inputs -->
    <div class="flex flex-col">
      <label class="flex text-gray-800 mb-2" for="name">
        Name </label>
        <input id="name" type="text" name="name"
class="border-2 border-solid border-blue-200
rounded px-2 py-1" />
      </div>
    </section>
</template>
```



Profile Card Generator

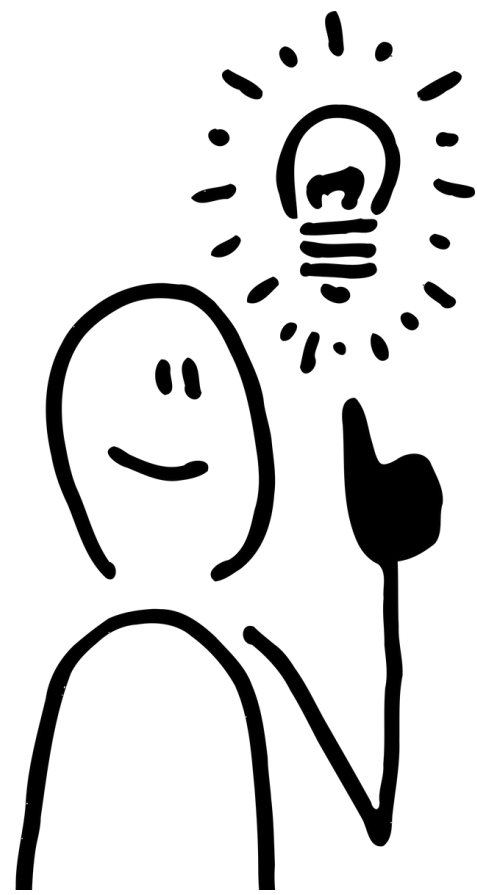
Name

Example: profile card editor

To keep track of the name input data, we will add a **two-way binding** to it using **v-model** and set a **name** property in the component's data initializer

```
<template>
<!-- rest of the template -->
  <input id="name" type="text"
    name="name" class="border-2 border-solid
    border-blue-200 rounded px-2 py-1" v-model="name" />
<!-- rest of the template -->
</template>

<script setup>
import { ref } from 'vue'
const emit = defineEmits(['submit'])
const name = ref('');
</script>
```



Example: profile card editor

We will also need a **submit** button that, upon being clicked, **sends the form data to the parent** by emitting a **submit event** with the form's contents:

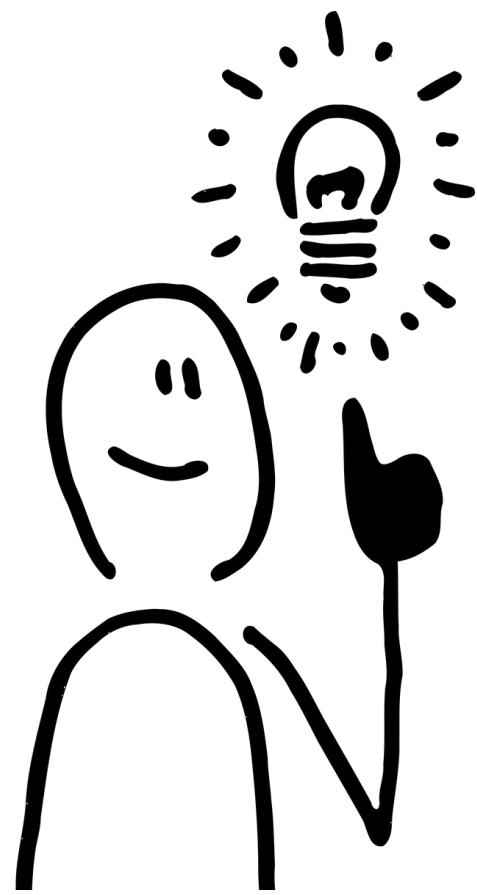
```
<template>
<!-- rest of template -->
<div class="flex flex-row mt-12">
  <button type="submit" @click="submitForm()">
    Submit</button>
</div>
<!-- rest of template -->
</template>
```

```
<script setup>
import { ref } from 'vue'
const emit = defineEmits(['submit'])
const name = ref('');
function submitForm() {
  emit('submit', {
    name: this.name
  });
}
</script>
```

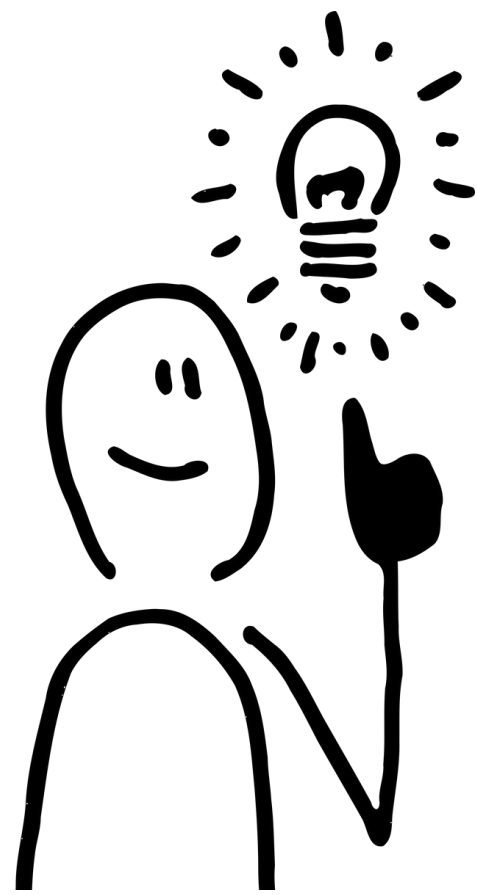
Profile Card Generator

Name

Submit



Example: profile card editor



The next step is to store the form's state in the **App component**.

To begin with, we will need a **formData** attribute returned from **reactive()**.

We will also need a way to update formData. Hence, we will add an **update(formData)** method

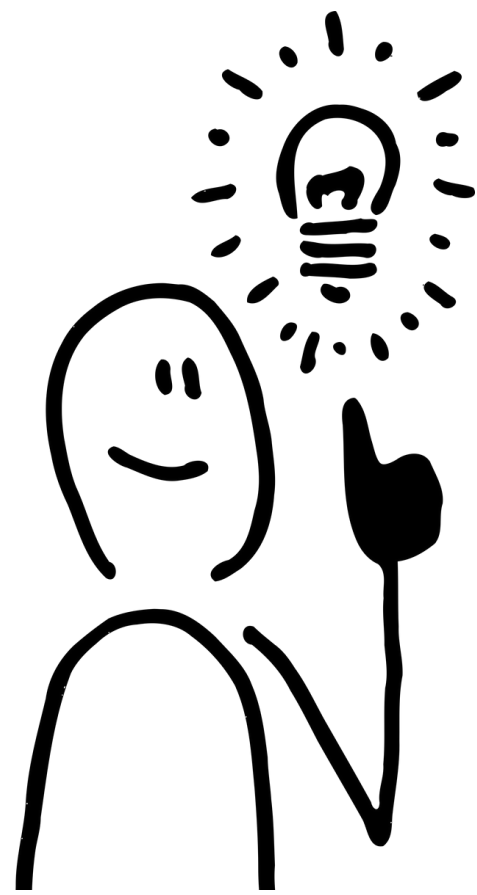
```
<script setup>
import AppHeader from '@components/AppHeader.vue'
import AppProfileDisplay from '@components/AppProfileDisplay.vue'
import AppProfileForm from '@components/AppProfileForm.vue'
import { reactive } from 'vue'

const formData = reactive({name:''});
function update() {
  formData.name = e.name;
}
</script>

<template>
  <!-- rest of template -->
  <AppProfileForm @submit="update($event)"/>
  <!--rest of template -->
</template>
```

Next, we need to bind `update()` to the `submit` event emitted by `AppProfileForm`. We will do this using the `@submit` shorthand and with the magic event object notation as `update($event)`:

Example: profile card editor



To display the name inside of **AppProfileDisplay**, we will need to add `formData` as a prop:

```
<script setup>
  const props = defineProps({formData:Object});
</script>
```

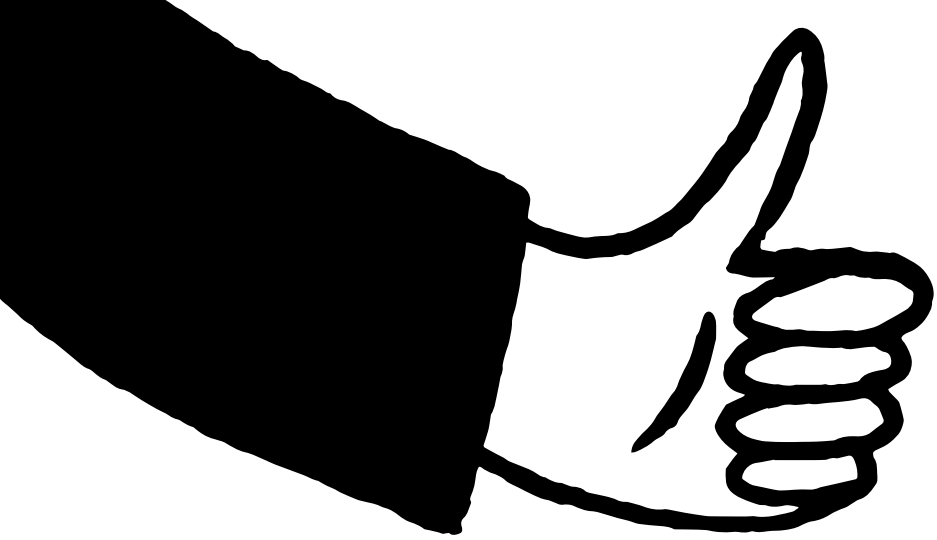
We will also need to display the name using `formData.name`.

```
<template>
  <section class="md:w-1/3 h-64 bg-blue-200 flex p-12">
    --- Profile Card ---
    <h3 class="font-bold font-lg">{{ formData.name }}</h3>
  </section>
</template>
```

App needs to pass `formData` to **AppProfileDisplay** as a prop:

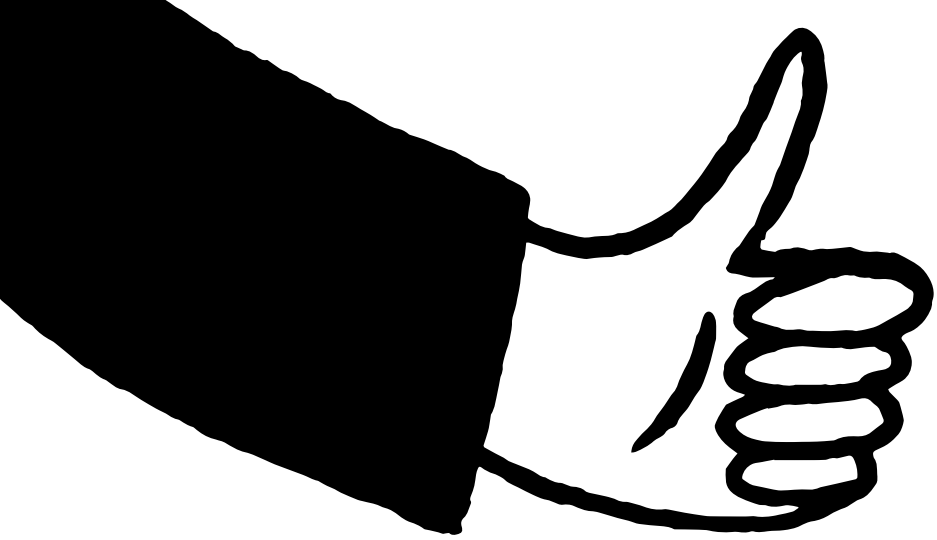
```
<template>
  <!-- rest of template -->
  <AppProfileDisplay :form-data="formData" />
  <!-- rest of template -->
</template>
```

Profile Card Generator	
Name John Doe	John Doe
Submit	



Exercise 9.01 – adding an occupation field to the profile card generator





Exercise 9.02 – adding a Clear button to the profile card generator



Adding simple state management

For our simple application, we can replace a lot of the boilerplate code if we use the **reactive()** API to build a simple store.

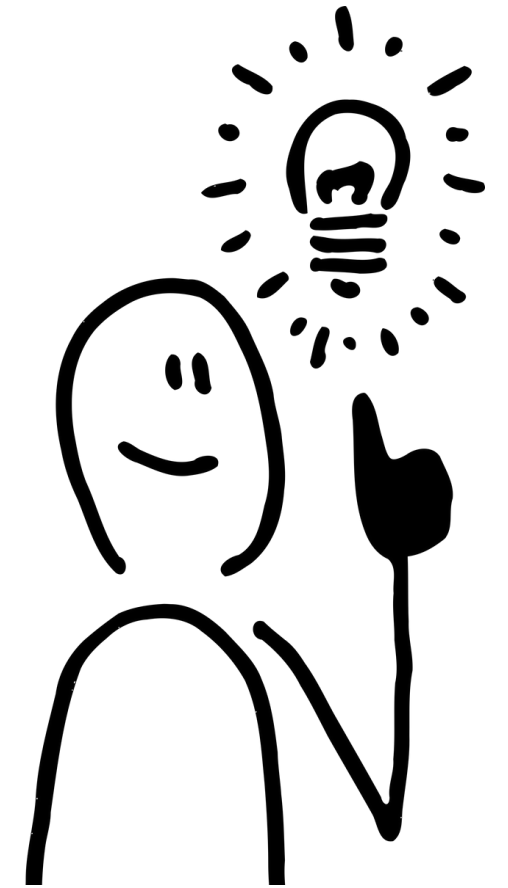
Let's start by building a new file, **store.js**, that uses a reactive object for our profile values:

```
import { reactive } from 'vue';  
export const store = reactive({  
  name: '',  
  occupation: ''  
});
```

This very simple object will be very powerful due to the use of Vue 3's reactivity support. Any component making use of the values from here will be able to rely on knowing that when a value changes, it will instantly be reflected. Right away, we can see how this simplifies things as we switch to the store

In **AppProfileForm**, let's import the store first:

```
<script setup>  
  import { store } from '@/store.js';  
</script>
```



Next, update both fields to **point to the store instead of local data**. In the following code, the v-model value was changed and the **Submit** button is removed – it's no longer necessary

```
<!-- rest of component -->
<div class="flex flex-col">
  <label class="flex text-gray-800 mb-2" for="name">
    Name </label>
    <input id="name" type="text" name="name"
      class="border-2 border-solid border-blue-200
        rounded px-2 py-1" v-model="store.name"/>
  </div>

  <div class="flex flex-col mt-2">
    <label class="flex text-gray-800 mb-2"
      for="occupation">Occupation</label>
    <input id="occupation" type="text" name="occupation"
      v-model="store.occupation"
      class="border-2 border-solid border-blue-200
        rounded px-2 py-1"/>
  </div>
```



Now, we can edit the script block to remove most of the previous logic. The **clear method** needs to be updated to change the store values:

```
<script setup>
import { store } from '@store.js';

function clear() {
  store.name = '';
  store.occupation = '';
}
</script>
```



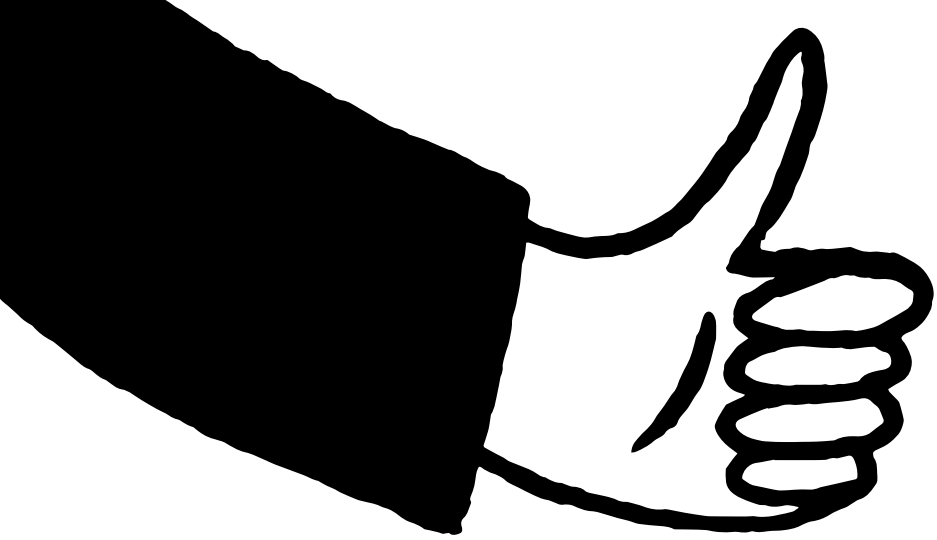
Next, we can make similar changes to **AppProfileDisplay**. First, import the store:

```
<script setup>
import { store } from '@store.js';
</script>
```

We've now switched from a system of events being broadcasted from components to one of a simpler, shared state.

Then, modify the template:

```
<template>
<section class="md:w-1/3 flex flex-col p-12">
  <!-- Profile Card -->
  <h3 class="font-bold font-lg">{{ store.name }}</h3>
  <p class="mt-2">{{ store.occupation }}</p>
</section>
</template>
```

**Exercise 9.03 –
moving the Clear button to the
application header profile card
generator and updating the
clear logic**





ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

State Management with Pinia

Chapter 10

Advanced Web Design

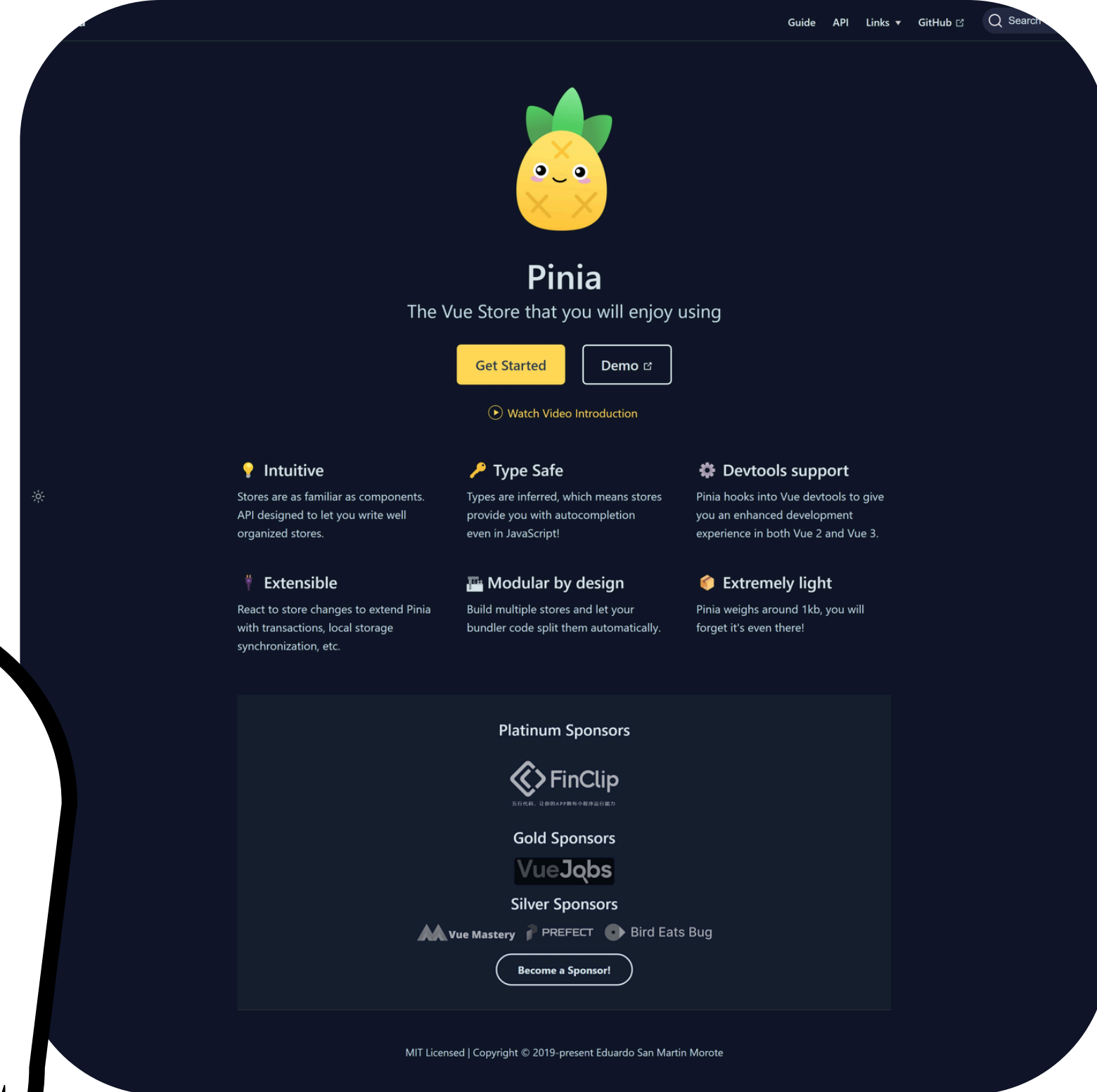
10 Intro

In the previous chapter, you were introduced to the idea of a state and how it can be used to synchronize data between multiple components in a Vue application. You first saw an example of handling states via event broadcasting and then improved upon that by including a simple state library.

In this chapter, you'll learn about the **Pinia** project and understand how it can help manage complex state interactions in your Vue applications.

- What Pinia is
- Installing Pinia
- Using Pinia to create a store
- Adding and using getters in your Pinia store
- Working with Pinia actions
- Debugging Pinia in Devtools

Pinia (<https://pinia.vuejs.org>) is a state management library for Vue.js applications. As you saw in Chapter 9, The State of Vue State Management, handling data that must be synchronized between multiple different components requires some form of management. Pinia helps with this by providing a simple method of setting up a **central store** that contains your **application's state**. Your components make use of this store to ensure they're all using the same information



Installing Pinia

```
npm init vue@latest
```

```
ray@Mandalore:~/projects/testingzone/vue3stuff$ npm init vue@latest  
  
Vue.js - The Progressive JavaScript Framework  
  
✓ Project name: ... vue-project  
✓ Add TypeScript? ... No / Yes  
✓ Add JSX Support? ... No / Yes  
✓ Add Vue Router for Single Page Application development? ... No / Yes  
? Add Pinia for state management? > No / Yes
```

existing Vue 3 application:

```
npm install pinia
```

main.js

```
import { createPinia } from 'pinia'
```

```
app.use(createPinia())
```

Using Pinia to create a store

1. Create a new Vue application and enable Pinia. This will give you a Vue application with a store already created. You will find it under **src/stores/counter.js**:

```
import { defineStore } from 'pinia'
export const useCounterStore = defineStore({
  id: 'counter',
  state: () => ({
    counter: 1
  }),
  getters: {
    doubleCount: (state) => state.counter * 2
  },
  actions: {
    increment() {
      this.counter++
    }
  }
})
```

When installed,
Pinia defined one piece of data,
counter, with a value of 1. How
can we access this in
our application?

Using Pinia to create a store

2. Switch to the **App.vue** file and remove all the contents. First, let's define the layout to simply output the value of counter

```
<template>
  <p>
    Counter: {{ store.counter }}
  </p>
</template>
```

3. Next, we need to make our store available to the component. We'll define this in a script setup block

```
<script setup>
  import { useCounterStore } from './stores/counter'
  const store = useCounterStore()
</script>
```

4. We begin by importing the store. Once imported, we create an instance of the store so that it can be used in our template.

Counter: 1

Using Pinia to create a store

5. To get any benefits from a shared state at all, we need at least one more component. In the components folder, create a new file, **EditCounter.vue**, and use the following short snippet of code

```
<script setup>
  import { useCounterStore } from '@stores/counter';
  const store = useCounterStore()
</script>

<template>
  <h2>Edit Counter</h2>
  <input type="text" v-model="store.counter">
</template>
```

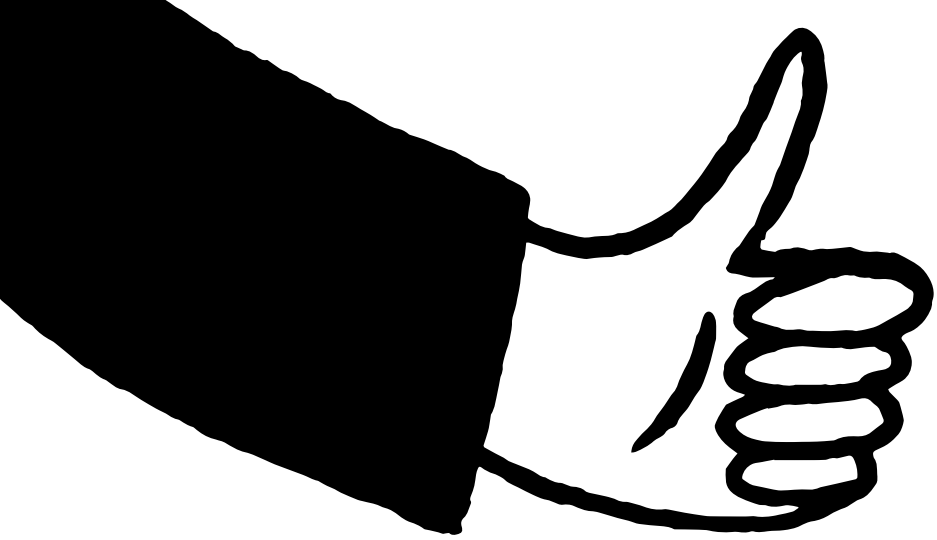
6. As with the **App.vue** component, we use the setup block to import the store and create an instance. This time, we use a simple **edit field** and **v-model** to bind its value to the store's counter value. Return to App.vue and edit it to import and use the EditCounter component

```
<script setup>
  import EditCounter from './components/EditCounter.vue'
  import { useCounterStore } from './stores/counter'
  const store = useCounterStore();
</script>

<template>
  <p>Counter: {{ store.counter }}</p>
  <EditCounter></EditCounter>
</template>
```

Counter: 6

Edit Counter



Exercise 10.01 – building a Color Preview app with a shared state

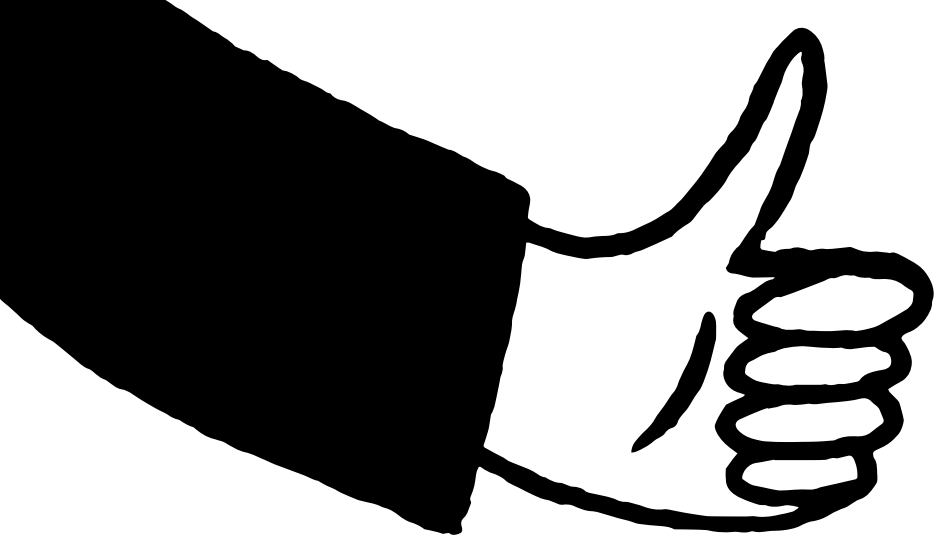


Adding and using getters in your Pinia store

As stated earlier, getters in Pinia act just like computed properties. They allow you to request a simple value that's generated by custom logic written in a function. If you go back to the original Pinia store created by default, you'll see it had a getter defined:

```
import { defineStore } from 'pinia'
export const useCounterStore = defineStore({
  id: 'counter',
  state: () => ({
    counter: 0
  }),
  getters: {
    doubleCount: (state) => state.counter * 2
  },
  // rest of file...
})
```

```
<template>
  <p>Counter: {{ store.counter }}</p>
  <p>Double Count: {{ store.doubleCount }}</p>
</template>
```



Exercise 10.02 – improving the Color Preview app with getters



Working with Pinia actions

Actions are the Pinia equivalent of component methods.

They let you define custom logic for a **store** and can be **asynchronous** as well.

This is useful for times when **server-side logic** needs to be called to validate a change to the state.

Actions are defined with the actions block of a Pinia object, and you can see an example in the default store created by Pinia.

```
import { defineStore } from 'pinia'
export const useCounterStore = defineStore({
  id: 'counter',
  state: () => ({
    counter: 0
  }),
  // rest of store...
  actions: {
    increment() {
      this.counter++
    }
  }
})
```

In this example, the increment action simply takes the counter value and adds one to it. Actions access state values by using the this scope and, as stated previously, can be asynchronous as well.

An example of an action with a bit of logic could look like so:

```
decrement() {
  if(this.counter > 0) this.counter--
}
```



Questions?

